РОССИЙСКАЯ АКАДЕМИЯ НАРОДНОГО ХОЗЯЙСТВА И ГОСУДАРСТВЕННОЙ СЛУЖБЫ

при Президенте Российской Федерации (РАНХиГС)

ОТЧЁТ

по индивидуальному заданию

«Разработка и интеграция программного модуля на C++»

Вариант 5. Калькулятор упущенной выгоды от оттока клиентов (Churn Leakage Estimator)

Выполнил: Дроздов Артём Игоревич, группа 2

Проверил: Марухленко А.Л.

2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ
ГЛАВА 1. ИССЛЕДОВАНИЕ ВХОДНОГО ПРОЕКТА
ГЛАВА 2. РАЗРАБОТКА ЛОГИКИ МОДУЛЯ
ГЛАВА 3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ
ГЛАВА 4. ТЕСТИРОВАНИЕ И ПУБЛИКАЦИЯ
ЗАКЛЮЧЕНИЕ
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ
ПРИЛОЖЕНИЯ

ВВЕДЕНИЕ

В работе исследуется готовый REST-сервер на C++17 (авторизация по Bearer-токенам, ролевая модель RBAC, хранение данных в SQLite) и в него встраивается расчётный модуль индивидуального варианта.

Цель работы — реализовать модуль «Калькулятор упущенной выгоды от оттока клиентов» (вариант 5), встроить его в сервер и проверить работу, а также исследовать сервер и выявить несоответствия реализации RBAC техническому заданию.

Задачи: изучить архитектуру сервера, собрать и запустить его; проверить эндпоинты; описать алгоритм и блок-схему модуля; реализовать функции parse, validate, calculate, to_json; встроить модуль через task.hpp; провести тестирование (контрольный пример, границы, ошибки, авторизация); сохранить проект в git.

ГЛАВА 1. ИССЛЕДОВАНИЕ ВХОДНОГО ПРОЕКТА

1.1. Основы REST-API

REST (Representational State Transfer) — архитектурный стиль, при котором клиент обращается к серверу по HTTP к ресурсам, заданным URL. Метод определяет действие: GET — чтение, POST — создание/вычисление, PUT — изменение, DELETE — удаление. Сервер не хранит состояние клиента между запросами: подлинность подтверждается токеном в заголовке Authorization: Bearer <token>. Данные передаются в формате JSON, результат запроса выражается кодом состояния HTTP (200, 201, 400, 401, 403, 404, 409, 500).

1.2. Структура проекта и назначение модулей

Файл	**Назначение**
main.cpp	Точка входа: инициализация БД, middleware (логирование, формат ошибок, CORS), регистрация маршрутов, запуск на порту 8080
src/json.hpp	Сборка JSON-ответов (JObj, JArr, экранирование jesc)
src/crypto.hpp	SHA-256, хеширование паролей с солью, генерация токенов
src/database.hpp	Работа с SQLite (db_run/db_all/db_row), схема БД, стартовые пользователи, конфигурация (TTL=300 с, лимит истории=20)
src/routes/auth.hpp	8 эндпоинтов авторизации и управления пользователями, проверка токена и ролей
src/routes/public.hpp	Счётчики, публичная история, логи, самотестирование
src/modules/example/stats.hpp	Демонстрационный модуль статистики (/works/calc)
src/modules/individual/task.hpp	Каркас индивидуального модуля (маршруты calc-individual и history-individual)
src/modules/individual/variant_mine.hpp	Модуль варианта 5 (разработан в данной работе)
src/lib/	Сторонние httplib.h (HTTP-движок) и загрузчик SQLite
frontend/index.html	Веб-коллекция запросов для ручного тестирования

1.3. Сборка и запуск проекта

Проект собирается без внешних зависимостей, кроме SQLite (на Windows используется встроенная winsqlite3.dll). Сборка выполнялась компилятором

GCC из дистрибутива w64devkit; предусмотрена также сборка через CMake. Команда сборки и запуска:

```
g++ -std=c++17 -O1 main.cpp -o rest_server.exe -lws2_32 -pthread
.\rest_server.exe
```

После старта сервер доступен на http://localhost:8080; при первом запуске создаются пользователи owner1, admin1, user1 (пароль совпадает с логином). Проверка работоспособности — GET /health.

1.4. Схема базы данных и роли пользователей

Таблица	Поля
users	id, username (UNIQUE), password (соль$SHA-256), role (user/admin/owner)
sessions	token (PK), user_id, expires_at
works	id, user_id, datetime, input_json, result_json
logs	id, datetime, status, message

Роли: user — обычный пользователь; admin — управляет пользователями группы user и просматривает логи; owner — полные права. Пользователя owner1 удалить нельзя. Токен живёт 300 секунд, при каждом запуске сервера таблица sessions очищается.

1.5. Обзор эндпоинтов и их назначение

Метод и путь	Доступ	Назначение
GET /health	все	Проверка работы сервера
GET /api/v1.1/counters	все	Счётчики: пользователи, расчёты, ошибки
GET /api/v1.1/history	все	Публичная история расчётов
POST /auth/register	все	Регистрация (роль user)
POST /auth/login	все	Вход, выдача токена
GET /auth/profile	токен	Профиль и оставшееся время токена
POST /auth/refresh	токен	Продление токена
POST /auth/logout	токен	Выход, удаление сессии
PUT /auth/users/password	токен	Смена пароля (себе — все; чужой — по правилам ролей)
GET /auth/users	admin, owner	Список пользователей
DELETE /auth/users	admin, owner	Удаление пользователя
GET /admin/stats	admin, owner	Системные логи
GET /admin/run-tests	admin, owner	Встроенные автотесты (20 проверок)
POST /works/calc, GET /works/history	токен	Демонстрационный модуль статистики
POST /works/calc-individual	токен	Индивидуальный модуль — вариант 5

GET /works/history-individual	токен	История расчётов варианта

1.6. Описание инструментов тестирования через web и curl

Для ручной проверки используется веб-коллекция frontend/index.html: в ней заданы шаблоны запросов ко всем эндпоинтам, токен после входа подставляется автоматически, а каждый запрос сохраняется в истории вместе с эквивалентной командой curl. Для автоматизации написаны скрипты PowerShell: demo_api.ps1 (демонстрация модуля) и test_rbac.ps1 (проверка прав и кодов ответов, 33 проверки), а также tests/http_tests.py (87 проверок на Python, только стандартная библиотека). Пример запроса curl:

```
curl.exe -X POST "http://localhost:8080/api/v1.1/works/calc-individual" -H "Content-Type:
application/json" -H "Authorization: Bearer <TOKEN>" -d
"{\"active_users_start\":5000,\"churned_users\":250,\"arpu\":30.0,\"winback_cost_per_user\":15.0}
"
```

1.7. Проверка корректности работы эндпоинтов

Все эндпоинты были вызваны под ролями user, admin, owner и без токена. Базовая функциональность (регистрация, вход, профиль, продление и завершение сессии, счётчики, история, расчёты) работает корректно. Для user запросы к административным эндпоинтам возвращают 403, что соответствует матрице доступа.

1.8. Описание выявленных несоответствий и ошибок

При сопоставлении поведения сервера с техническим заданием выявлены следующие несоответствия (пп. 1, 2, 6 подтверждены запросами к исходной версии, пп. 3–5 выявлены анализом кода). Все они исправлены.

№	Несоответствие	Где	Исправление
1	GET и DELETE /auth/users без токена возвращали 403 вместо 401 (проверка токена и роли объединены в одно условие)	auth.hpp	Разделены проверки: нет токена — 401, недостаточно прав — 403. То же в /admin/stats
2	В логах /admin/stats поле datetime пустое: write_log не записывал время	database.hpp	write_log записывает now_iso()
3	admin мог менять пароль owner и других	auth.hpp	Для роли admin разрешены

	admin; по ТЗ — только себе и группе user		цели с ролью user и он сам; owner — любые
4	admin мог удалять пользователей любой роли (кроме owner1)	auth.hpp	admin удаляет только группу user (иначе 403); при удалении чистятся сессии
5	Смена пароля несуществующего пользователя возвращала 200 «Password updated»	auth.hpp	Возвращается 404 User not found
6	Глобальный обработчик ошибок затирал тело ответа 400 модуля: вместо {"error":"..."} приходило «Request failed» (нарушение п. 4.3 ТЗ)	main.cpp	Если тело ответа уже задано, оно сохраняется

Замечания, не требовавшие правки в рамках работы: SHA-256 с солью не защищает от быстрого перебора (для продакшена рекомендуется bcrypt/argon2); нет ограничения частоты запросов и HTTPS; разбор JSON выполняется поиском подстроки, поэтому вложенные структуры не поддерживаются; CORS открыт для всех доменов.

1.9. Запуск встроенных автотестов

Эндпоинт GET /admin/run-tests выполняет 20 проверок и доступен только admin/owner. Набор был дополнен проверками модуля варианта 5: 6 системных проверок (БД, пароли, токены, конфигурация), 4 корректных данных, 4 граничных значения, 4 неверных данных и 2 неполных данных. Результат — 20 из 20. Отдельно добавлен эндпоинт GET /admin/run-tests-individual (admin/owner): 20 проверок только модуля варианта без обращения к БД; он доступен в веб-интерфейсе кнопкой «20 тестов варианта 5» (рисунок 1).

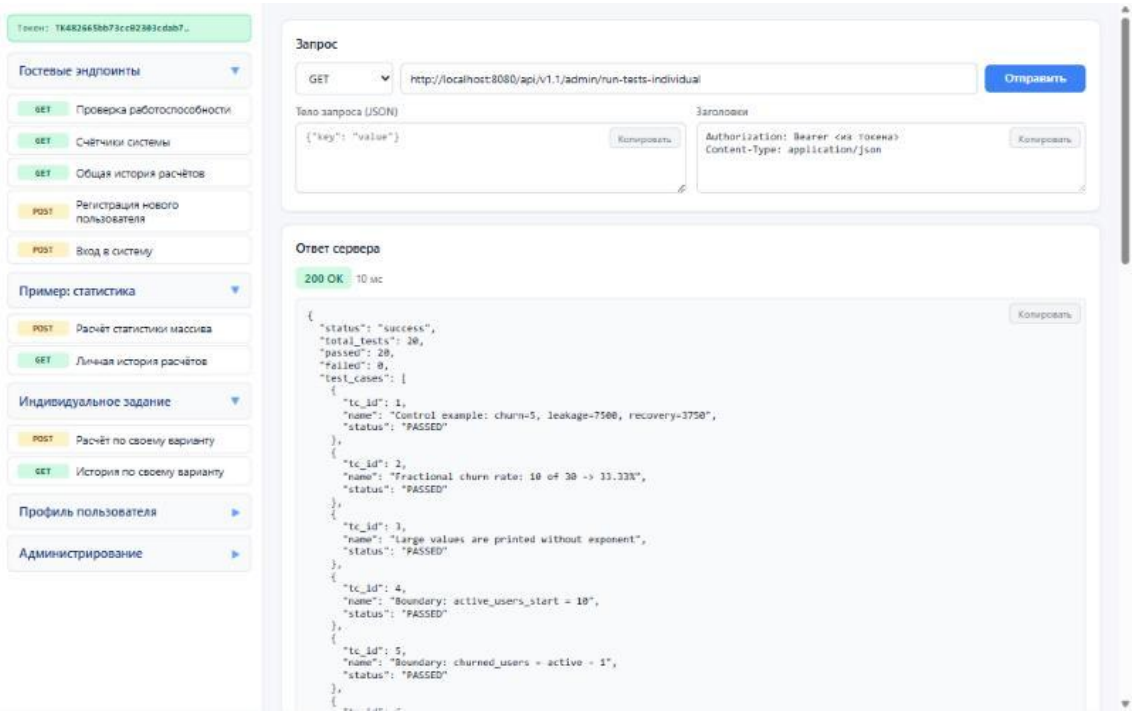


Рисунок 1 — 20 тестов варианта 5 в веб-интерфейсе (admin1)

```
GET /api/v1.1/admin/run-tests (admin1)

status=success total=20 passed=20 failed=0
 1. DB handle active                          PASSED
 2. Users table accessible                    PASSED
 3. Password verify accepts valid             PASSED
 4. Password verify rejects invalid           PASSED
 5. Token unique, prefix TK                   PASSED
 6. Config: TTL > 0, history limit > 0        PASSED
 7. V5 sample: churn_rate = 5.0               PASSED
 8. V5 sample: leakage = 7500.0               PASSED
 9. V5 sample: recovery budget = 3750.0       PASSED
10. V5 valid input accepted                   PASSED
11. V5 boundary: active_users_start = 10      PASSED
12. V5 boundary: churned = active - 1         PASSED
13. V5 boundary: churned = 0 -> zeros         PASSED
14. V5 boundary: arpu = 0.01 accepted         PASSED
15. V5 invalid: active_users_start = 9        PASSED
16. V5 invalid: churned = active              PASSED
17. V5 invalid: churned < 0                   PASSED
18. V5 invalid: arpu = 0 / winback < 0        PASSED
19. V5 incomplete: empty body                 PASSED
20. V5 incomplete: missing arpu               PASSED
```

Рисунок 2 — Результат встроенных автотестов (/admin/run-tests)

ГЛАВА 2. РАЗРАБОТКА ЛОГИКИ МОДУЛЯ

2.1. Постановка задачи

Вариант 5. Калькулятор упущенной выгоды от оттока клиентов (Churn Leakage Estimator). Тема: проектирование системы мониторинга финансовых потерь в подписочных сервисах.

Параметр	Тип	Описание	Ограничение
active_users_start	int	Активные подписчики на начало месяца	>= 10
churned_users	int	Ушедшие подписчики за месяц	0 <= x < active_users_start
arpu	double	Средний доход с одного подписчика в месяц	> 0.0
winback_cost_per_user	double	Стоимость возврата одного пользователя	> 0.0

Формулы:

```
churn_rate_percentage   = (churned_users / active_users_start) * 100.0
monthly_revenue_leakage = churned_users * arpu
total_recovery_budget   = churned_users * winback_cost_per_user
```

Контрольный пример: вход active_users_start = 5000, churned_users = 250, arpu = 30.0, winback_cost_per_user = 15.0; выход churn_rate_percentage = 5.0, monthly_revenue_leakage = 7500.0, total_recovery_budget = 3750.0.

2.2. Словесные алгоритмы

Алгоритм обработки запроса:

- Проверить токен авторизации; при отсутствии или просрочке вернуть 401.
- Передать тело запроса в parse(), получить структуру Input.
- Выполнить validate(); при ошибке вернуть 400 и {"error": "<описание>"}.
- Выполнить calculate(), сформировать JSON функцией to_json().
- Сохранить запрос и результат в таблицу works (user_id, datetime, input_json, result_json).
- Вернуть 200 и JSON с тремя показателями.

Алгоритм разбора (parse):

- Тело запроса разбирается как строгий плоский JSON-объект: '{', затем пары "ключ": число через запятую, затем '}'.
- Неизвестный ключ — ошибка «Unknown field: <ключ>»; повторяющийся — «Duplicate field: <ключ>».
- Число читается функцией strtod; нечисловое значение (в том числе строка в кавычках), NaN и Inf — «<ключ> must be a number».
- Мусор после закрывающей скобки или нарушение синтаксиса — «Invalid JSON ...» / «Unexpected content ...».
- Отсутствующее поле — «<ключ> is required»; дробное значение в int-поле — «<ключ> must be an integer».
- Первая найденная ошибка сохраняется в parse_error и возвращается валидатором (в отличие от шаблона, здесь отсутствующее поле не превращается молча в 0).

Алгоритм валидации (validate), порядок проверок:

- parse_error не пустой — вернуть его;
- active_users_start < 10 — «active_users_start must be >= 10»;
- churned_users < 0 — «churned_users must be >= 0»;
- churned_users >= active_users_start — «churned_users must be < active_users_start»;
- arpu <= 0 (или NaN) — «arpu must be > 0.0»;
- winback_cost_per_user <= 0 — «winback_cost_per_user must be > 0.0»;
- иначе вернуть пустую строку (данные корректны).

Алгоритм расчёта (calculate): вычислить churn_rate_percentage, затем monthly_revenue_leakage, затем total_recovery_budget по формулам п. 2.1 и вернуть структуру Output.

2.3. Блок-схемы алгоритмов

Блок-схема обработки запроса приведена на рисунке 3, блок-схема расчётной части — на рисунке 4 (оформление по ГОСТ 19.701-90: терминатор, ввод-вывод, процесс, решение).

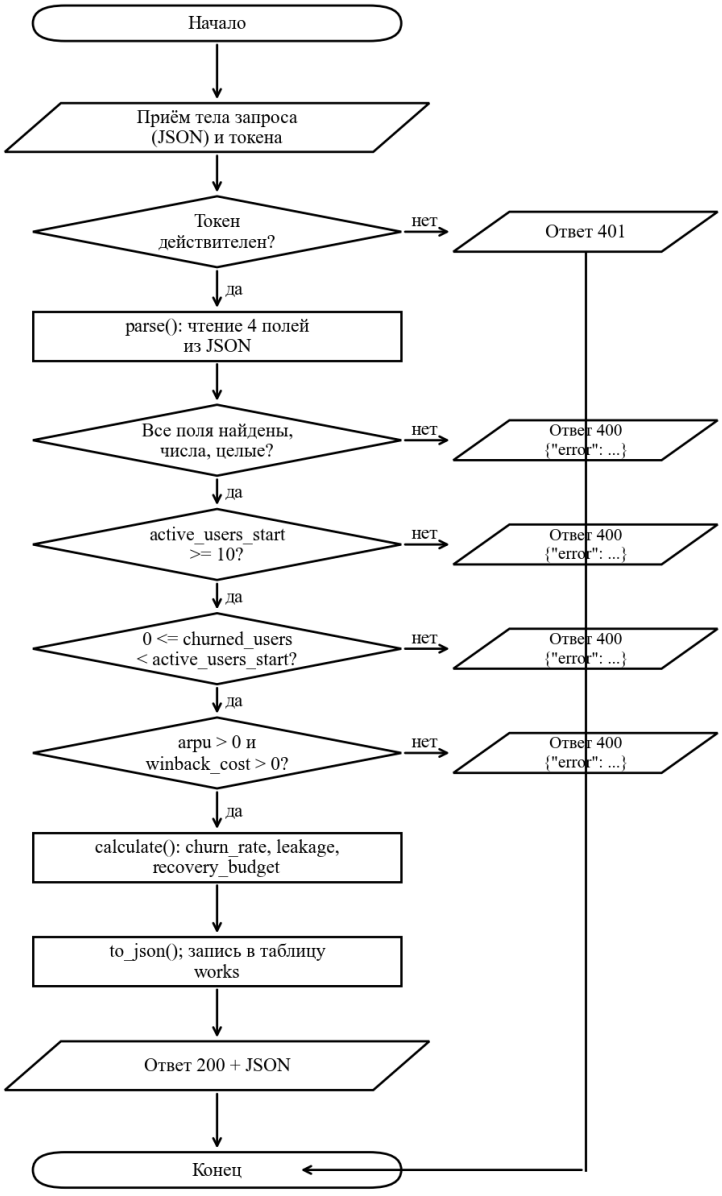


Рисунок 3 — Блок-схема обработки запроса модулем

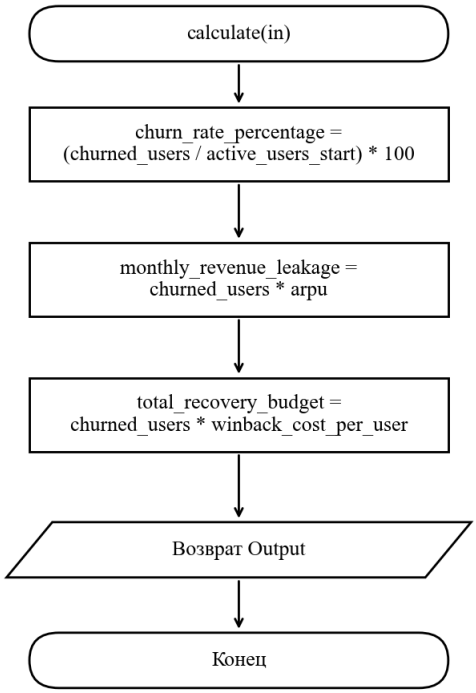


Рисунок 4 — Блок-схема расчётной части (calculate)

ГЛАВА 3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

3.1. Выбор конструкций C++ и библиотек

Использован стандарт C++17 и только стандартная библиотека (string, cmath, cstdlib, cstdio), как требует ТЗ; для сборки ответа применяется готовый JObj из json.hpp. Целые поля хранятся в long long — это исключает переполнение и позволяет сравнивать значения без преобразований; вещественные — в double. Разбор чисел выполняется strtod, так как он сообщает, было ли прочитано число, и не бросает исключений. Числа в ответе форматируются через snprintf(%.10g), чтобы большие значения не выводились в экспоненциальной записи.

3.2. Реализация модуля варианта

Модуль находится в файле src/modules/individual/variant_mine.hpp, пространство имён TaskVariant. Он содержит структуры Input и Output и функции parse, validate, calculate, to_json. Полный листинг — в приложении А.

3.3. Встраивание модуля в систему

В файле task.hpp строка #include "variant_roi.hpp" заменена на #include "variant_mine.hpp". Маршруты POST /api/v1.1/works/calc-individual и GET /api/v1.1/works/history-individual остались прежними: каркас вызывает TaskVariant::parse → validate → calculate → to_json и сохраняет результат в таблицу works. Если запись в БД не удалась, возвращается 500 с {"error": "Could not save result"} вместо ложного успеха. Автотесты варианта вынесены в variant_tests.hpp и используются эндпоинтом /admin/run-tests-individual, встроенными автотестами (public.hpp) и консольной программой tests/variant_tests.cpp.

ГЛАВА 4. ТЕСТИРОВАНИЕ И ПУБЛИКАЦИЯ

4.1. Методика тестирования

Применялось три уровня проверки: (1) модульные тесты модуля без сервера — tests/variant_tests.cpp, 20 проверок; (2) встроенный блок автотестов сервера — /admin/run-tests, 20 проверок; (3) сквозные запросы к работающему серверу — demo_api.ps1, test_rbac.ps1 (33 проверки) и tests/http_tests.py (87 проверок, запуск на пустой БД), а также ручные вызовы через web-интерфейс и curl. Ручной расчёт контрольных примеров выполнен заранее и сравнён с результатом программы.

4.2. Контрольный пример

Ручной расчёт: churn_rate = 250 / 5000 × 100 = 5,0 %; leakage = 250 × 30,0 = 7500,0; recovery_budget = 250 × 15,0 = 3750,0. Сервер вернул {"churn_rate_percentage":5,"monthly_revenue_leakage":7500,"total_recovery_budget":3750} — результат совпал.

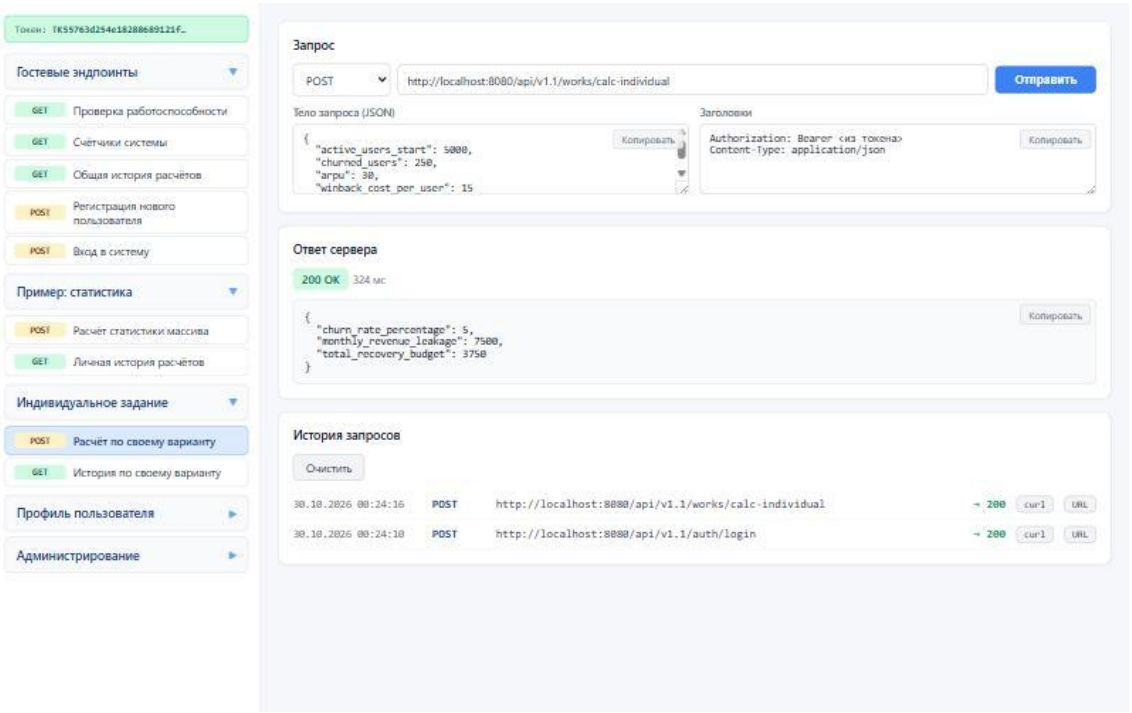


Рисунок 5 — Контрольный пример в веб-интерфейсе (200 OK)

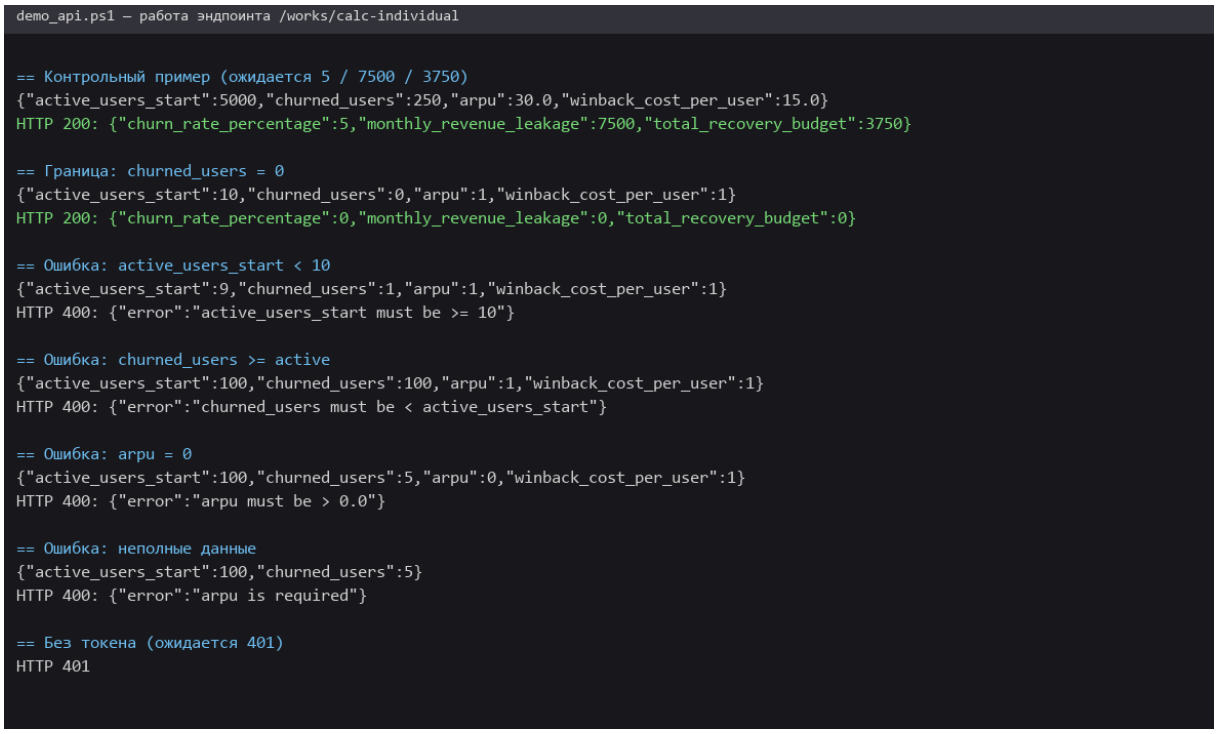


Рисунок 6 — Контрольный пример, границы и ошибки через API

4.3. Граничные значения

Вход (active, churned, arpu, winback)	**Ручной расчёт (churn %, leakage, budget)**	**Результат**
5000, 250, 30.0, 15.0	5; 7500; 3750	совпал
1000, 100, 20, 5	10; 2000; 500	совпал
10, 0, 5, 5 (churned = 0)	0; 0; 0	совпал, 200
10, 9, 1, 1 (churned = active - 1)	90; 9; 9	совпал, 200
10, 1, 0.01, 0.01 (минимальные arpu/winback)	10; 0.01; 0.01	совпал, 200

4.4. Невалидные данные

Вход	**Ожидание**	**Результат**
active_users_start = 9	400, «active_users_start must be >= 10»	400, совпало
churned_users = active_users_start	400, «churned_users must be < active_users_start»	400, совпало
churned_users = -1	400	400
arpu = 0	400, «arpu must be > 0.0»	400, совпало
winback_cost_per_user = -3	400	400
нет поля arpu	400, «arpu is required»	400, совпало
пустое тело запроса	400	400
active_users_start = 10.5	400 (поле целое)	400
лишнее поле в JSON	400, «Unknown field: ...»	400, совпало
повторяющееся поле	400, «Duplicate field: ...»	400, совпало

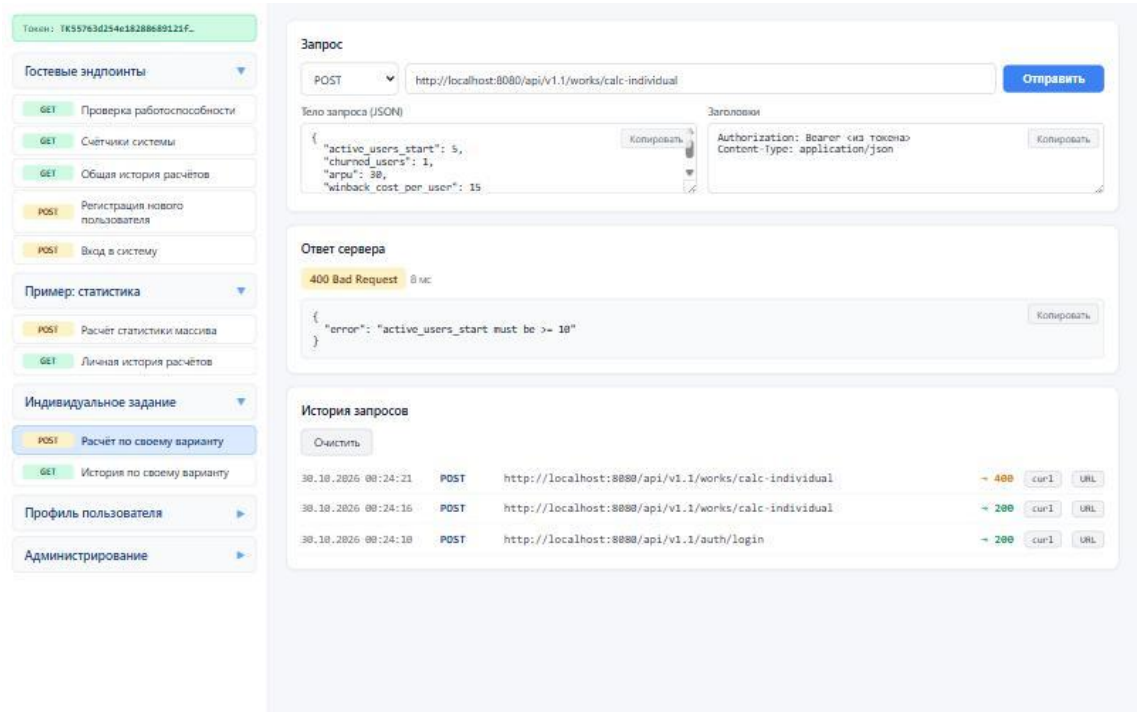


Рисунок 7 — Ошибка валидации в веб-интерфейсе (400 Bad Request)

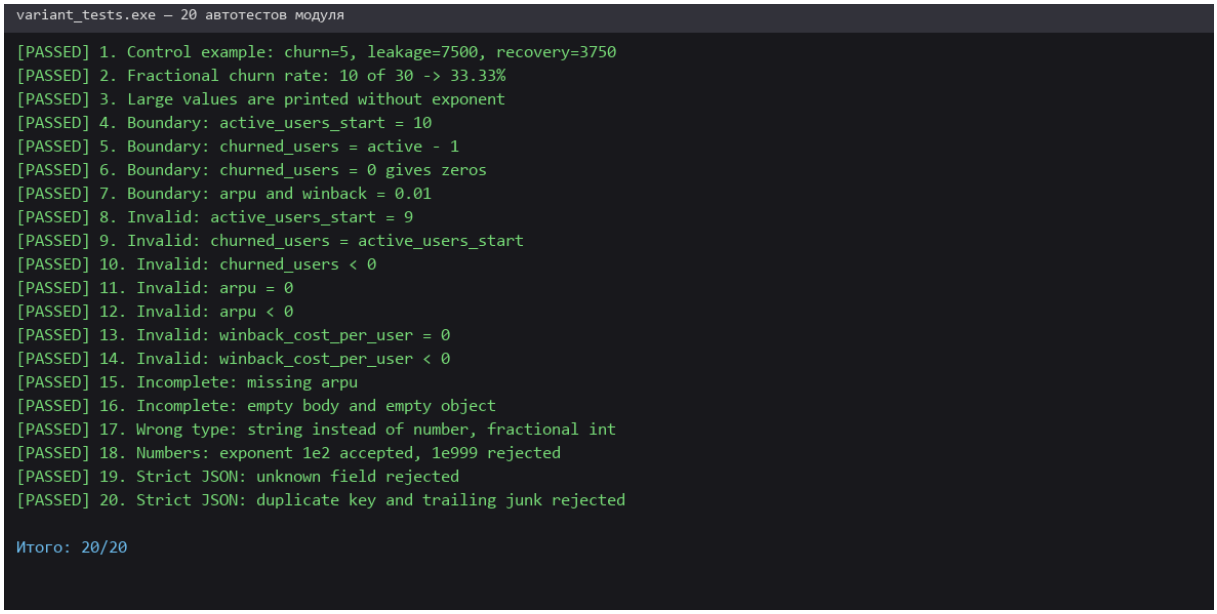


Рисунок 8 — Модульные тесты (20 из 20)

4.5. Проверка авторизации

Скрипт test_rbac.ps1 проверяет вход, регистрацию, права ролей user/admin/owner, смену пароля, удаление пользователей, защиту owner1, работу

с токеном после logout и доступ к индивидуальному модулю без токена. Все 33 проверки пройдены (в том числе исправленные ошибки из п. 1.8). Дополнительно tests/http_tests.py выполняет 87 HTTP-проверок (все пройдены), результаты сохранены в tests/http_results.json.

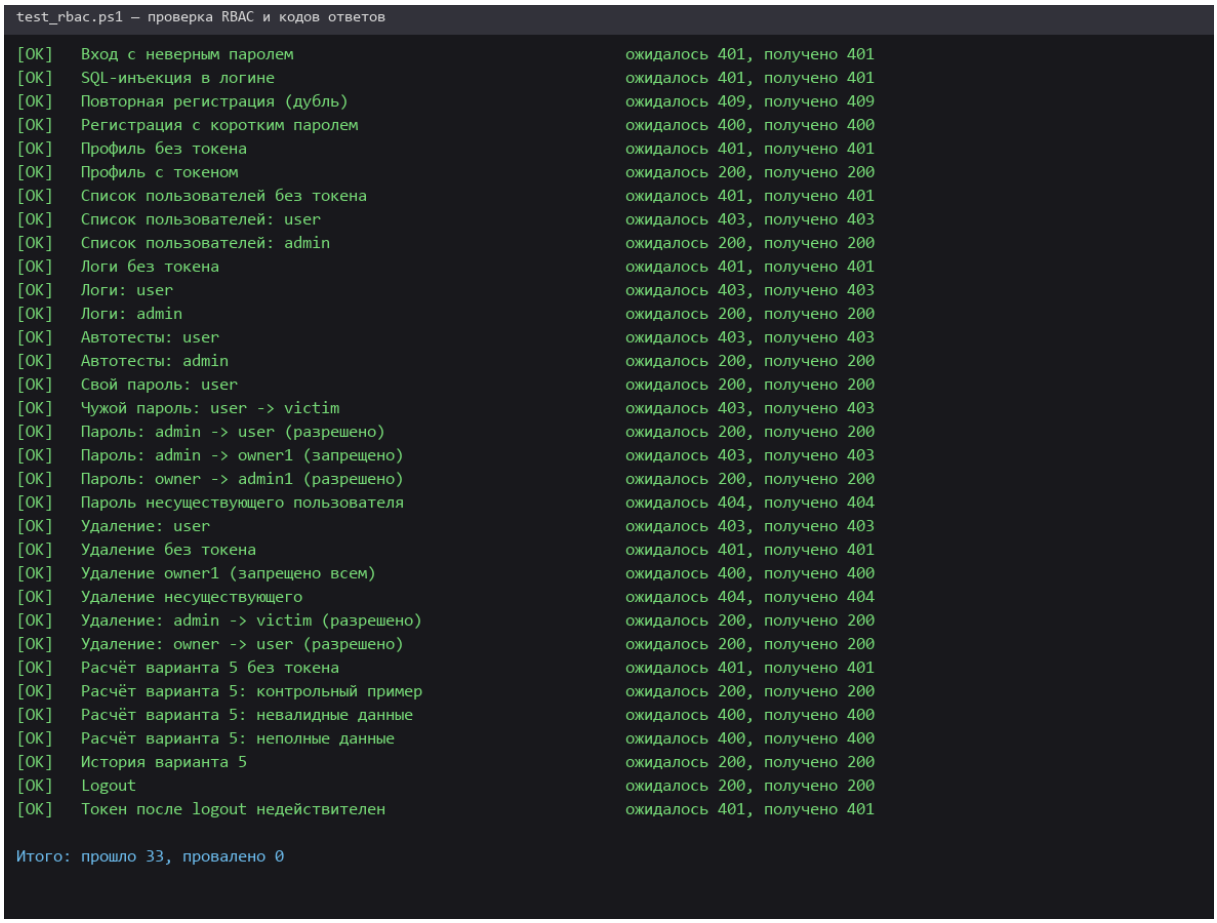


Рисунок 9 — Проверка RBAC и кодов ответов

4.6. Публикация в git-репозиторий

Проект сохраняется в git командами:

```
git init
git add .
git commit -m "Variant 5: Churn Leakage Estimator + RBAC fixes"
git remote add origin <URL репозитория>
git push -u origin main
```

Ссылка на репозиторий: https://github.com/Hix0r/rest-c

ЗАКЛЮЧЕНИЕ

Изучен и запущен REST-сервер на C++; описаны структура, база данных, роли и эндпоинты. Выявлено и исправлено шесть несоответствий ТЗ (коды 401/403, права admin, пустая дата в логах, 404 при смене пароля, затирание тела ошибки). Реализован и встроен модуль варианта 5 с полной валидацией входных данных. Контрольный пример и граничные значения подтверждены ручным расчётом; 20 модульных тестов, 20 встроенных автотестов, 33 проверки RBAC и 87 HTTP-проверок пройдены.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Репозиторий проекта: https://github.com/mal-corp/rest-c
2. cpp-httplib: https://github.com/yhirose/cpp-httplib
3. SQLite Documentation: https://sqlite.org/docs.html
4. ISO/IEC 14882:2017 — Programming languages — C++17.
5. ГОСТ 19.701-90. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения.
6. RFC 9110. HTTP Semantics.

ПРИЛОЖЕНИЕ А. Листинг variant_mine.hpp

```
#pragma once
// ============================================================================
// ВАРИАНТ 5: Калькулятор упущенной выгоды от оттока клиентов
//            (Churn Leakage Estimator).
//
// Входные поля JSON (строгий плоский объект, все поля обязательны):
//   active_users_start    (int)    >= 10
//   churned_users         (int)    0 <= x < active_users_start
//   arpu                  (double) > 0
//   winback_cost_per_user (double) > 0
//
// Формулы:
//   churn_rate_percentage     = churned_users / active_users_start * 100
//   monthly_revenue_leakage   = churned_users * arpu
//   total_recovery_budget     = churned_users * winback_cost_per_user
// ============================================================================


#include <string>
#include <cmath>
#include <cstdlib>
#include <cstdio>
#include "../../json.hpp"


namespace TaskVariant {


    struct Input {
        long long active_users_start;
        long long churned_users;
        double    arpu;
        double    winback_cost_per_user;


        // Пустая строка = JSON разобран без ошибок. Иначе validate()
        // вернёт это сообщение (-> HTTP 400).
        std::string parse_error;
    };


    struct Output {
        double churn_rate_percentage;
        double monthly_revenue_leakage;
        double total_recovery_budget;
    };


    // ----------------------------------------------------------------------
    // Проверка входных данных: "" если всё ок, иначе текст ошибки.
    // ----------------------------------------------------------------------
    inline std::string validate(const Input& in) {
        if (!in.parse_error.empty())
            return in.parse_error;
        if (in.active_users_start < 10)
            return "active_users_start must be >= 10";
        if (in.churned_users < 0)
            return "churned_users must be >= 0";
        if (in.churned_users >= in.active_users_start)
            return "churned_users must be < active_users_start";
        if (!(in.arpu > 0.0))
            return "arpu must be > 0.0";
        if (!(in.winback_cost_per_user > 0.0))
```

```
            return "winback_cost_per_user must be > 0.0";
        return "";
    }

    inline Output calculate(const Input& in) {
        Output out{};
        out.churn_rate_percentage =
            ((double)in.churned_users / (double)in.active_users_start) * 100.0;
        out.monthly_revenue_leakage = (double)in.churned_users * in.arpu;
        out.total_recovery_budget   = (double)in.churned_users * in.winback_cost_per_user;
        return out;
    }

    // ---------------------------------------------------------------------
    // Строгий разбор плоского JSON-объекта из четырёх числовых полей.
    // Ошибкой считаются: не JSON-объект, неизвестное или повторяющееся поле,
    // нечисловое значение, NaN/Inf, отсутствующее поле, дробное значение
    // в int-поле, мусор после закрывающей скобки.
    // ---------------------------------------------------------------------
    inline Input parse(const std::string& body) {
        Input in{};
        static const char* const keys[4] = {
            "active_users_start", "churned_users", "arpu", "winback_cost_per_user"
        };
        double val[4] = {0, 0, 0, 0};
        bool   seen[4] = {false, false, false, false};

        size_t pos = 0;
        auto ws = [&] {
            while (pos < body.size() && (body[pos] == ' ' || body[pos] == '\t' ||
                                         body[pos] == '\r' || body[pos] == '\n')) ++pos;
        };
        auto fail = [&](const std::string& e) { in.parse_error = e; return in; };

        ws();
        if (pos >= body.size() || body[pos] != '{') return fail("Expected JSON object");
        ++pos; ws();

        if (pos < body.size() && body[pos] != '}') {
            while (true) {
                if (pos >= body.size() || body[pos] != '"') return fail("Invalid JSON: expected
field name");
                size_t e = body.find('"', pos + 1);
                if (e == std::string::npos) return fail("Invalid JSON: unterminated field name");
                std::string key = body.substr(pos + 1, e - pos - 1);
                pos = e + 1; ws();
                if (pos >= body.size() || body[pos] != ':') return fail("Invalid JSON: expected
':'");
                ++pos; ws();

                int idx = -1;
                for (int i = 0; i < 4; ++i) if (key == keys[i]) idx = i;
                if (idx < 0)     return fail("Unknown field: " + key);
                if (seen[idx])   return fail("Duplicate field: " + key);

                size_t b = pos;
                while (pos < body.size() && std::string("+-.0123456789eE").find(body[pos]) !=
std::string::npos) ++pos;
                std::string tok = body.substr(b, pos - b);
                if (tok.empty() || tok[0] == '+') return fail(key + " must be a number");
```

```
                char* end = nullptr;
                double v = std::strtod(tok.c_str(), &end);
                if (end != tok.c_str() + tok.size() || !std::isfinite(v))
                    return fail(key + " must be a number");
                val[idx] = v; seen[idx] = true;

                ws();
                if (pos < body.size() && body[pos] == ',') { ++pos; ws(); continue; }
                break;
            }
        }
        if (pos >= body.size() || body[pos] != '}') return fail("Invalid JSON: expected '}'");
        ++pos; ws();
        if (pos != body.size()) return fail("Unexpected content after JSON object");

        for (int i = 0; i < 4; ++i)
            if (!seen[i]) return fail(std::string(keys[i]) + " is required");
        for (int i = 0; i < 2; ++i)
            if (val[i] != std::floor(val[i]) || std::fabs(val[i]) > 9e15)
                return fail(std::string(keys[i]) + " must be an integer");

        in.active_users_start    = (long long)val[0];
        in.churned_users         = (long long)val[1];
        in.arpu                  = val[2];
        in.winback_cost_per_user = val[3];
        return in;
    }

    // Число без экспоненты (JObj::num печатает 1.2e+06 для больших значений).
    inline std::string fmt(double v) {
        char buf[64];
        std::snprintf(buf, sizeof buf, "%.10g", v);
        std::string s = buf;
        if (s.find('e') != std::string::npos) {
            std::snprintf(buf, sizeof buf, "%.2f", v);
            s = buf;
        }
        return s;
    }

    inline std::string to_json(const Output& out) {
        JObj o;
        o.raw("churn_rate_percentage",   fmt(out.churn_rate_percentage));
        o.raw("monthly_revenue_leakage", fmt(out.monthly_revenue_leakage));
        o.raw("total_recovery_budget",   fmt(out.total_recovery_budget));
        return o.str();
    }

} // namespace TaskVariant
```

ПРИЛОЖЕНИЕ Б. Листинг task.hpp

```
#pragma once
// ============================================================================
// МОДУЛЬ INDIVIDUAL: каркас роута.
// Не содержит математики — только вызовы функций варианта.
// ============================================================================

#include <string>
#include "../../lib/httplib.h"
#include "../../database.hpp"
#include "../../json.hpp"
#include "../../routes/auth.hpp"

// ----------------------------------------------------------------------------
// ПОДКЛЮЧИТЕ СВОЙ ВАРИАНТ ЗДЕСЬ (замените одну строку).
// ----------------------------------------------------------------------------
// #include "variant_roi.hpp"
#include "variant_mine.hpp"
#include "variant_tests.hpp"

namespace Individual {
    using namespace std;
    using namespace httplib;

    inline void init_routes(Server& svr) {

        // ─── GET /admin/run-tests-individual (admin/owner) ──────────────────
        // 20 проверок модуля варианта без обращения к БД.
        svr.Get("/api/v1.1/admin/run-tests-individual", [](const Request& req, Response& res) {
            string role, name;
            if (!Auth::auth(req, role, name)) { res.status = 401; return; }
            if (!Auth::admin(role))           { res.status = 403; return; }

            const auto tests = VariantTests::run();
            int passed = 0;
            JArr cases;
            for (size_t i = 0; i < tests.size(); ++i) {
                if (tests[i].second) ++passed;
                JObj o;
                o.int_("tc_id", (long long)(i + 1));
                o.str("name", tests[i].first);
                o.str("status", tests[i].second ? "PASSED" : "FAILED");
                cases.add(o.str());
            }
            JObj rep;
            rep.str("status", passed == (int)tests.size() ? "success" : "failed");
            rep.int_("total_tests", (long long)tests.size());
            rep.int_("passed", passed);
            rep.int_("failed", (long long)tests.size() - passed);
            rep.raw("test_cases", cases.str());
            res.set_content(rep.str(), "application/json");
        });

        // ─── POST /works/calc-individual ────────────────────────────────────
        svr.Post("/api/v1.1/works/calc-individual", [](const Request& req, Response& res) {
            string role, name;
            int uid = Auth::auth(req, role, name);
            if (!uid) { res.status = 401; return; }
```

```
            auto in = TaskVariant::parse(req.body);

            string err = TaskVariant::validate(in);
            if (!err.empty()) {
                JObj o; o.str("error", err);
                res.status = 400;
                res.set_content(o.str(), "application/json");
                return;
            }

            auto out = TaskVariant::calculate(in);
            string result = TaskVariant::to_json(out);

            if (!db_run("INSERT INTO works (user_id,datetime,input_json,result_json) "
                        "VALUES (?,?,?,?);",
                        {to_string(uid), now_iso(), req.body, result})) {
                JObj o; o.str("error", "Could not save result");
                res.status = 500;
                res.set_content(o.str(), "application/json");
                return;
            }
            res.set_content(result, "application/json");
        });

        // ── GET /works/history-individual ──────────────────────────────
        svr.Get("/api/v1.1/works/history-individual", [](const Request& req, Response& res) {
            string role, name;
            int uid = Auth::auth(req, role, name);
            if (!uid) { res.status = 401; return; }

            int limit = MAX_HISTORY_LIMIT;
            if (req.has_param("limit")) {
                try { limit = min(stoi(req.get_param_value("limit")), limit); }
                catch (...) {}
            }

            JArr arr;
            for (auto& r : db_all(
                "SELECT id,datetime,input_json,result_json FROM works "
                "WHERE user_id=" + to_string(uid) +
                " ORDER BY id DESC LIMIT " + to_string(limit) + ";")) {
                JObj o;
                o.int_("id", stoll(r[0]));
                o.str("datetime", r[1]);
                o.raw("input", r[2]);
                o.raw("result", r[3]);
                arr.add(o.str());
            }
            res.set_content(arr.str(), "application/json");
        });
    }
}
```

ПРИЛОЖЕНИЕ В. Блок-схема алгоритма расчётной части

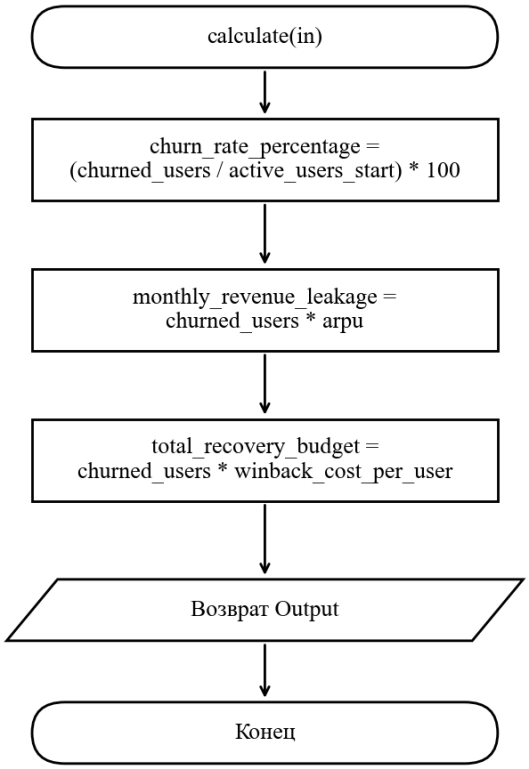


Рисунок В.1 — Блок-схема функции calculate

ПРИЛОЖЕНИЕ Г. Скриншоты тестирования

Результаты тестирования приведены на рисунках 1, 2 и 5–9 основной части отчёта.

ПРИЛОЖЕНИЕ Д. Дополнительные материалы

Модульные тесты — variant_tests.hpp и tests/variant_tests.cpp; скрипты demo_api.ps1, test_rbac.ps1 и tests/http_tests.py; инструкция по запуску — README_VARIANT_5.md и ЗАПУСК.md.

```
#pragma once
// ============================================================================
// 20 автоматических проверок варианта 5.
// Используются: GET /api/v1.1/admin/run-tests-individual и tests/variant_tests.cpp.
// Не обращаются к БД и сети.
// ============================================================================

#include <string>
#include <vector>
#include <utility>
#include <cmath>
#include "variant_mine.hpp"


namespace VariantTests {

    inline std::vector<std::pair<std::string, bool>> run() {
        using namespace TaskVariant;


        auto body = [](const std::string& a, const std::string& c,
                       const std::string& r, const std::string& w) {
            return "{\"active_users_start\":" + a + ",\"churned_users\":" + c +
                   ",\"arpu\":" + r + ",\"winback_cost_per_user\":" + w + "}";
        };
        auto err   = [](const std::string& b) { return validate(parse(b)); };
        auto valid = [&](const std::string& b) { return err(b).empty(); };
        auto approx = [](double a, double b) { return std::fabs(a - b) < 1e-9; };
        const std::string sample = body("5000", "250", "30.0", "15.0");
        auto ref = calculate(parse(sample));
        auto zero = calculate(parse(body("10", "0", "5", "5")));


        return {
            // --- корректные данные ---
            {"Control example: churn=5, leakage=7500, recovery=3750",
                valid(sample) && approx(ref.churn_rate_percentage, 5.0) &&
                approx(ref.monthly_revenue_leakage, 7500.0) && approx(ref.total_recovery_budget,
3750.0)},
            {"Fractional churn rate: 10 of 30 -> 33.33%",
                approx(calculate(parse(body("30", "10", "1", "1"))).churn_rate_percentage, 100.0
/ 3)},
            {"Large values are printed without exponent",
                to_json(calculate(parse(body("100000000", "50000000", "1000", "1")))).find("e+")
== std::string::npos},
            // --- граничные значения ---
            {"Boundary: active_users_start = 10", valid(body("10", "0", "1", "1"))},
            {"Boundary: churned_users = active - 1", valid(body("10", "9", "1", "1"))},
            {"Boundary: churned_users = 0 gives zeros",
                zero.churn_rate_percentage == 0.0 && zero.monthly_revenue_leakage == 0.0 &&
zero.total_recovery_budget == 0.0},
            {"Boundary: arpu and winback = 0.01", valid(body("10", "1", "0.01", "0.01"))},
            // --- неверные данные ---
            {"Invalid: active_users_start = 9", !valid(body("9", "1", "1", "1"))},
```

```
            {"Invalid: churned_users = active_users_start", !valid(body("100", "100", "1",
"1"))},
            {"Invalid: churned_users < 0", !valid(body("100", "-1", "1", "1"))},
            {"Invalid: arpu = 0", !valid(body("100", "5", "0", "1"))},
            {"Invalid: arpu < 0", !valid(body("100", "5", "-2", "1"))},
            {"Invalid: winback_cost_per_user = 0", !valid(body("100", "5", "1", "0"))},
            {"Invalid: winback_cost_per_user < 0", !valid(body("100", "5", "1", "-3"))},
            // --- неполные и некорректные данные ---
            {"Incomplete: missing arpu",

err("{\"active_users_start\":100,\"churned_users\":5,\"winback_cost_per_user\":2}") == "arpu is
required"},
            {"Incomplete: empty body and empty object", !valid("") && !valid("{}")},
            {"Wrong type: string instead of number, fractional int",
                !valid(body("\"abc\"", "5", "1", "1")) && !valid(body("10.5", "1", "1", "1"))},
            {"Numbers: exponent 1e2 accepted, 1e999 rejected",
                valid(body("1e2", "5", "1", "1")) && !valid(body("100", "5", "1e999", "1"))},
            {"Strict JSON: unknown field rejected",

!valid("{\"active_users_start\":100,\"churned_users\":5,\"arpu\":1,\"winback_cost_per_user\":1,\"
x\":1}")},
            {"Strict JSON: duplicate key and trailing junk rejected",

!valid("{\"active_users_start\":100,\"active_users_start\":100,\"churned_users\":5,\"arpu\":1,\"w
inback_cost_per_user\":1}") &&
                !valid(sample + "junk")},
        };
    }
}
// Автотесты модуля варианта 5 (20 проверок).
// Сборка: g++ -std=c++17 tests/variant_tests.cpp -o variant_tests
#include <iostream>
#include "../src/modules/individual/variant_tests.hpp"


int main() {
    auto tests = VariantTests::run();
    int passed = 0;
    for (size_t i = 0; i < tests.size(); ++i) {
        if (tests[i].second) ++passed;
        std::cout << (tests[i].second ? "[PASSED] " : "[FAILED] ") << (i + 1) << ". " <<
tests[i].first << "\n";
    }
    std::cout << "\nИтого: " << passed << "/" << tests.size() << "\n";
    return passed == (int)tests.size() ? 0 : 1;
}
```